

Production Deployment Guide: Domain & SSL Configuration for Docker Apps

This documentation outlines the steps required to take a Dockerized application running locally on a server and expose it securely to the internet using a custom domain, an Nginx reverse proxy, and Let's Encrypt SSL.

Prerequisites

- A registered domain name (e.g., janmasethu.com).
- Access to your domain registrar's DNS management console (e.g., GoDaddy).
- An Ubuntu server with Nginx and Certbot installed.
- A Docker container actively running your application on a specific port (e.g., localhost:5100).

Step 1: Point the Domain to Your Server (DNS Configuration)

Before the server can generate an SSL certificate, internet traffic for your domain must be routed to your server's public IP address.

1. Log in to your domain registrar and navigate to **DNS Management**.
2. Create or edit two **A Records** to point to your server's IP address (e.g., 72.61.228.9):

Type	Name (Host)	Value (Points To)	Purpose
A	@	72.61.228.9	Routes the root domain (janmasethu.com)
A	www	72.61.228.9	Routes the www subdomain (www.janmasethu.com)

Note: DNS changes can take a few minutes to propagate across the internet. Wait about 5-10 minutes before proceeding to the SSL step to avoid verification failures.

Step 2: Create the Nginx Reverse Proxy Configuration

Nginx acts as a traffic director. It will listen for web traffic on your domain's standard web ports (80/443) and forward it to your internal Docker container.

1. Open a new Nginx configuration file:
`sudo nano /etc/nginx/sites-available/janmasethu`
2. Paste the following configuration, ensuring the proxy_pass points to your Docker container's internal port (5100):

Nginx

```
server {  
    # Listen for both the root domain and the www version  
    server_name janmasethu.com www.janmasethu.com;  
  
    # --- RULE 1: ASSETS (The "No White Screen" Guarantee) ---  
    location ~* \.(?:css|js|map|jpe?g|gif|png|ico|svg|woff|woff2|ttf|eot)$ {  
        proxy_pass http://localhost:5100;  
  
        proxy_http_version 1.1;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
  
        add_header Cache-Control "public, max-age=604800";  
    }  
  
    # --- RULE 2: APP ROUTES (The "Refresh Fix") ---  
    location / {  
        proxy_pass http://localhost:5100;  
  
        # Intercept 404s and send index.html for smooth reloading  
        proxy_intercept_errors on;  
        error_page 404 =200 /;  
  
        proxy_http_version 1.1;  
        proxy_set_header Upgrade $http_upgrade;  
        proxy_set_header Connection 'upgrade';  
    }  
}
```

```
proxy_set_header Host $host;  
proxy_set_header X-Real-IP $remote_addr;  
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
}  
}
```

3. Save the file and exit your editor.

Step 3: Enable the Site and Test Nginx

With the configuration written, you must now safely activate it on the server.

1. Create the Shortcut (Symlink)

```
sudo ln -s /etc/nginx/sites-available/janmasethu /etc/nginx/sites-enabled/
```

- **What it does:** Tells Nginx this specific site configuration should be turned on.
- **Under the hood:** Nginx's main configuration file is programmed to include all files found within the sites-enabled directory. This command creates a symbolic link from sites-available to sites-enabled, allowing you to toggle sites on and off without deleting their code.

2. Test the Configuration (Pre-flight Validation)

```
sudo nginx -t
```

- **What it does:** Scans your new configuration file for typos or human errors.
- **Under the hood:** Runs the Nginx master process in "test mode" to parse the syntax for lexical errors or broken logic. This prevents a bad configuration file from crashing your live server.

3. Apply the Changes (Zero-Downtime Deployment)

```
sudo systemctl reload nginx
```

- **What it does:** Makes your new website live without dropping current visitors.
- **Under the hood:** Sends a SIGHUP (Signal Hang Up) to the Nginx master process. Nginx safely spins up new worker processes with the updated config and gracefully shuts down the old workers only after they finish serving their current active connections.

Step 4: Generate the SSL Certificate

Finally, secure the traffic with Let's Encrypt to provide users with a secure connection.

Run the Certbot Command:

```
sudo certbot --nginx -d janmasethu.com -d www.janmasethu.com
```

- **What it does:** Proves you own the domain, downloads encryption keys, automatically configures Nginx for HTTPS, and secures the site.
- **Under the hood:** Certbot uses the ACME protocol to perform an HTTP-01 Challenge. Let's Encrypt asks Certbot to place a specific cryptographic token at a specific URL on your domain. Because you passed the --nginx flag, Certbot temporarily manipulates Nginx to serve that token. Once verified, Let's Encrypt issues an X.509 certificate. Certbot then automatically injects the necessary TLS directives directly into your Nginx configuration file and reloads the server.

+1

Important: If Certbot prompts you to redirect HTTP traffic to HTTPS, select **"2"** (**Redirect**) to ensure all users are forced onto the secure connection.

Verification

Once Certbot completes successfully, navigate to <https://janmasethu.com> in your browser. The application running on your Docker container at port 5100 will now be securely accessible to the world.

Would you like me to draft a similar technical documentation covering how to package and run that initial Docker container on port 5100?